

Tugas 13: Praktikum Mandiri 13

Fatih Dzakwan Susilo - 0110224235

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

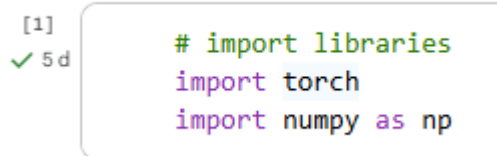
E-mail: fatihdzakwansusilo@gmail.com

1. Praktikum Mandiri

Link Github : https://github.com/FatihDzkwn/Project_Machine_Learning.git

Link Dataset : <https://www.kaggle.com/code/fgiorgio/multi-layer-perceptron-mnist>

1.1 Persiapan dan Import Library



The image shows a Jupyter Notebook cell with the following code:

```
[1] ✓ 5 d
# import libraries
import torch
import numpy as np
```

Gambar 1. Persiapan dan Import Library.

Kode Python ini merupakan tahap Persiapan dan Import Library. Pada tahap ini, programmer mengimpor dua library fundamental untuk komputasi ilmiah dan machine learning, yaitu PyTorch (torch) dan NumPy (numpy as np). PyTorch digunakan untuk membangun dan melatih model neural network dengan dukungan GPU acceleration, sedangkan NumPy menyediakan fungsi-fungsi untuk operasi array dan matriks multidimensi. Kedua library ini menjadi fondasi awal sebelum melakukan pemrosesan data, pembangunan model, atau komputasi numerik lainnya.

1.2 Persiapan dan Pemuatan Dataset MNIST

```
[2] ✓ 15 d
from torchvision import datasets
import torchvision.transforms as transforms

# number of subprocesses to use for data loading
num_workers = 0
# how many samples per batch to load
batch_size = 20

# convert data to torch.FloatTensor
transform = transforms.ToTensor()

# choose the training and test datasets
train_data = datasets.MNIST(root='data', train=True,
                             download=True, transform=transform)
test_data = datasets.MNIST(root='data', train=False,
                             download=True, transform=transform)

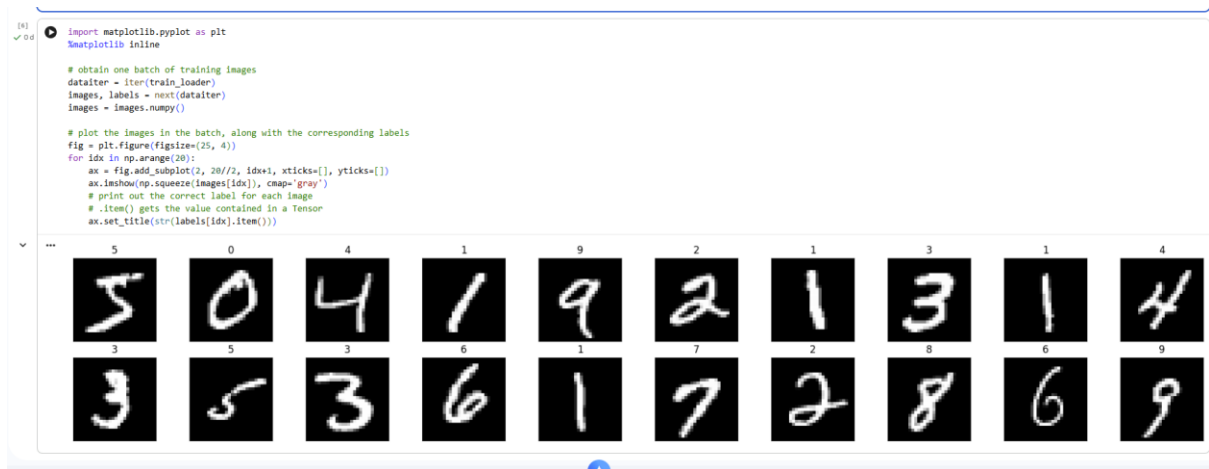
# prepare data loaders
train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
                                             num_workers=num_workers)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size,
                                           num_workers=num_workers)

100%|██████████| 9.91M/9.91M [00:01<00:00, 6.08MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 161kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.51MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 5.54MB/s]
```

Gambar 2. Persiapan dan Pemuatan Dataset MNIST.

Kode ini merupakan tahap Persiapan dan Pemuatan Dataset MNIST. Pada tahap ini, dataset MNIST yang berisi gambar tulisan tangan angka 0-9 diunduh dan dimuat menggunakan torchvision, kemudian ditransformasi menjadi tensor PyTorch agar dapat diproses oleh model neural network. Dataset dibagi menjadi data training (60.000 gambar) dan data testing (10.000 gambar), lalu diatur dalam DataLoader dengan ukuran batch 20 sampel per iterasi. DataLoader ini memfasilitasi proses pelatihan dengan menyediakan data secara bertahap dalam batch-batch kecil, sehingga model dapat belajar secara efisien tanpa memuat seluruh dataset sekaligus ke memori.

1.3 Visualisasi Data Training



Gambar 3. Visualisasi Data Training.

Kode ini merupakan tahap Visualisasi Data Training. Pada tahap ini, satu batch berisi 20 gambar diambil dari `train_loader` untuk divisualisasikan menggunakan `matplotlib`, dengan tujuan memverifikasi bahwa data telah dimuat dengan benar dan memahami karakteristik dataset. Setiap gambar ditampilkan dalam format grayscale bersama dengan label angka yang sesuai di atas gambar, disusun dalam grid 2 baris dan 10 kolom. Proses visualisasi ini penting sebagai tahap eksplorasi awal untuk memastikan transformasi data berjalan dengan baik dan memahami seperti apa bentuk input yang akan diproses oleh model neural network nantinya.

1.4 Visualisasi Detail Nilai Pixel

[7]

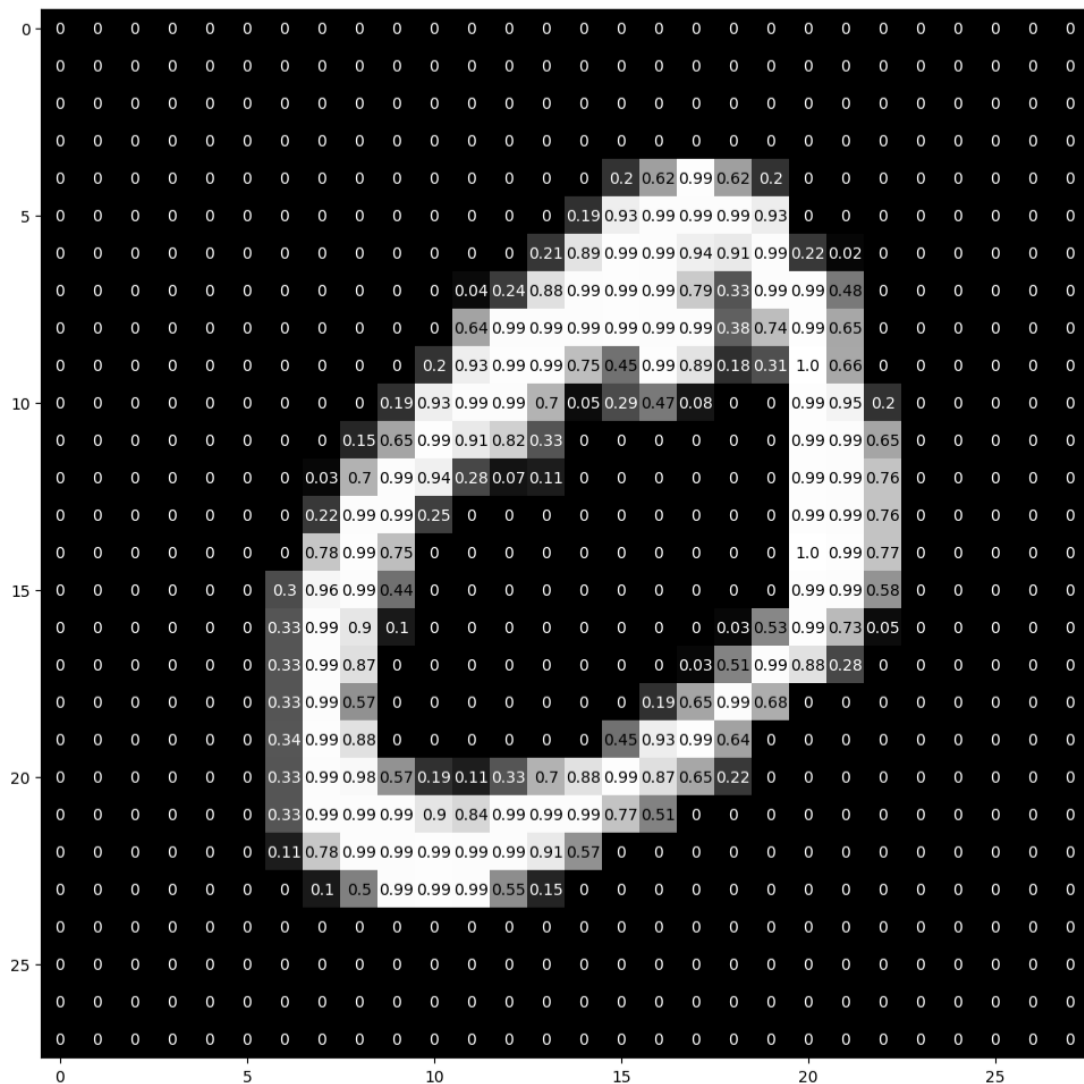
```

img = np.squeeze(images[1])

fig = plt.figure(figsize = (12,12))
ax = fig.add_subplot(111)
ax.imshow(img, cmap='gray')
width, height = img.shape
thresh = img.max()/2.5
for x in range(width):
    for y in range(height):
        val = round(img[x][y],2) if img[x][y] !=0 else 0
        ax.annotate(str(val), xy=(y,x),
                    horizontalalignment='center',
                    verticalalignment='center',
                    color='white' if img[x][y]<thresh else 'black')

```

v



Gambar 4. Visualisasi Detail Nilai Pixel.

Kode ini merupakan tahap **Visualisasi Detail Nilai Pixel**. Pada tahap ini, satu gambar tunggal (gambar dengan indeks 1) dari batch diambil dan ditampilkan secara detail dengan menampilkan nilai intensitas setiap pixel di atas gambar tersebut. Setiap pixel dalam gambar 28x28 akan dianotasi dengan nilai numeriknya yang berkisar antara 0 (hitam) hingga 1 (putih), dengan warna teks yang disesuaikan agar tetap terbaca (putih untuk area gelap, hitam untuk area terang). Visualisasi ini berguna untuk memahami representasi numerik dari gambar dan bagaimana

intensitas pixel membentuk pola angka tulisan tangan, sehingga kita dapat melihat secara langsung bagaimana data gambar sebenarnya disimpan sebagai matriks nilai-nilai numerik.

1.5 Definisi dan Inisialisasi Arsitektur Neural Network

```
[8]
✓ Od
import torch.nn as nn
import torch.nn.functional as F

## TODO: Define the NN architecture
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # linear layer (784 -> 1 hidden node)
        self.fc1 = nn.Linear(28 * 28, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 10)
        self.dropout = nn.Dropout(p=0.2)

    def forward(self, x):
        # flatten image input
        x = x.view(-1, 28 * 28)
        # add hidden layer, with relu activation function
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.dropout(F.relu(self.fc2(x)))
        x = self.fc3(x)
        return x

# initialize the NN
model = Net()
print(model)

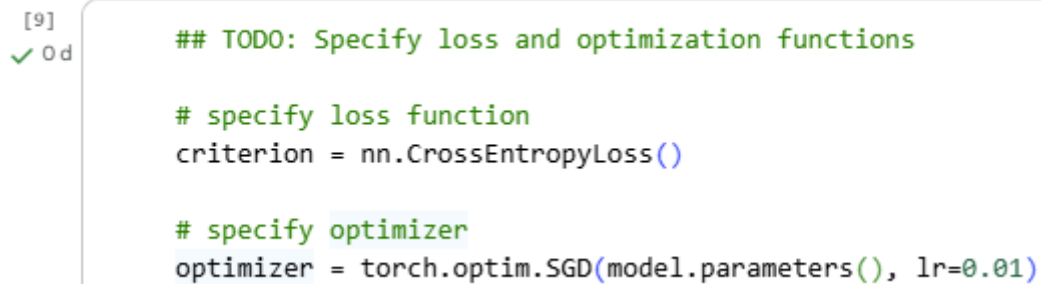
Net(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=512, bias=True)
  (fc3): Linear(in_features=512, out_features=10, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Gambar 5. Definisi dan Inisialisasi Arsitektur Neural Network.

Kode ini merupakan tahap Definisi dan Inisialisasi Arsitektur Neural Network. Pada tahap ini, sebuah class bernama Net didefinisikan yang mewarisi `nn.Module` dari PyTorch, dengan arsitektur fully connected neural network yang terdiri dari tiga layer linear (784 input nodes untuk gambar 28x28 pixel, dua hidden layer masing-masing 512 nodes, dan 10 output nodes untuk klasifikasi angka 0-9). Setiap hidden layer menggunakan fungsi aktivasi ReLU dan dropout sebesar 20% untuk mencegah overfitting, sementara input gambar akan diratakan (flatten) dari matriks 28x28 menjadi vector 784 elemen sebelum diproses. Setelah class didefinisikan, model

diinisialisasi dan strukturnya dicetak untuk memverifikasi arsitektur yang telah dibuat, sehingga siap untuk tahap training selanjutnya.

1.6 Konfigurasi Loss Function dan Optimizer



```
[9] ✓ 0d
## TODO: Specify loss and optimization functions

# specify loss function
criterion = nn.CrossEntropyLoss()

# specify optimizer
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Gambar 6. Konfigurasi Loss Function dan Optimizer.

Kode ini merupakan tahap Konfigurasi Loss Function dan Optimizer. Pada tahap ini, fungsi loss `CrossEntropyLoss` dipilih sebagai kriteria untuk mengukur seberapa jauh prediksi model dari label sebenarnya, yang merupakan pilihan standar untuk masalah klasifikasi multi-class seperti pengenalan angka MNIST. Optimizer `Stochastic Gradient Descent (SGD)` dikonfigurasi dengan learning rate 0.01 untuk memperbarui bobot-bobot model selama proses training, di mana optimizer ini akan meminimalkan nilai loss dengan menyesuaikan parameter model secara iteratif. Kedua komponen ini menjadi mesin penggerak proses pembelajaran model, di mana loss function mengukur performa dan optimizer mengarahkan perbaikan model menuju solusi yang lebih baik.

1.7 Training Model Neural Network

[10]
✓ 10m

```
# number of epochs to train the model
n_epochs = 30 # suggest training between 20-50 epochs

model.train() # prep model for training

for epoch in range(n_epochs):
    # monitor training loss
    train_loss = 0.0

    #####
    # train the model #
    #####
    for data, target in train_loader:
        # clear the gradients of all optimized variables
        optimizer.zero_grad()
        # forward pass: compute predicted outputs by passing inputs to the model
        output = model(data)
        # calculate the loss
        loss = criterion(output, target)
        # backward pass: compute gradient of the loss with respect to model parameters
        loss.backward()
        # perform a single optimization step (parameter update)
        optimizer.step()
        # update running training loss
        train_loss += loss.item()*data.size(0)

    # print training statistics
    # calculate average loss over an epoch
    train_loss = train_loss/len(train_loader.dataset)

    print('Epoch: {} \tTraining Loss: {:.6f}'.format(
        epoch+1,
        train_loss
    ))
```

✓

Epoch: 1	Training Loss: 0.823310
Epoch: 2	Training Loss: 0.322803
Epoch: 3	Training Loss: 0.250808
Epoch: 4	Training Loss: 0.203796
Epoch: 5	Training Loss: 0.171959
Epoch: 6	Training Loss: 0.147948
Epoch: 7	Training Loss: 0.130088
Epoch: 8	Training Loss: 0.116282
Epoch: 9	Training Loss: 0.104733
Epoch: 10	Training Loss: 0.095185
Epoch: 11	Training Loss: 0.087870
Epoch: 12	Training Loss: 0.079546
Epoch: 13	Training Loss: 0.073525
Epoch: 14	Training Loss: 0.068580
Epoch: 15	Training Loss: 0.063819
Epoch: 16	Training Loss: 0.058736
Epoch: 17	Training Loss: 0.055842
Epoch: 18	Training Loss: 0.050778
Epoch: 19	Training Loss: 0.048320
Epoch: 20	Training Loss: 0.045207
Epoch: 21	Training Loss: 0.043722
Epoch: 22	Training Loss: 0.040968
Epoch: 23	Training Loss: 0.038702
Epoch: 24	Training Loss: 0.036541
Epoch: 25	Training Loss: 0.035394
Epoch: 26	Training Loss: 0.032116
Epoch: 27	Training Loss: 0.031070
Epoch: 28	Training Loss: 0.028897
Epoch: 29	Training Loss: 0.027854
Epoch: 30	Training Loss: 0.027527

Gambar 7. Training Model Neural Network.

Kode ini merupakan tahap Training Model Neural Network. Pada tahap ini, model dilatih selama 30 epoch dengan mengiterasi seluruh dataset training, di mana setiap epoch terdiri dari proses forward pass untuk menghitung prediksi, menghitung loss antara prediksi dan target sebenarnya, backward pass untuk menghitung gradien, dan parameter update menggunakan optimizer. Dalam setiap iterasi batch, gradien direset terlebih dahulu dengan `zero_grad()`, kemudian loss dihitung dan dipropagasi mundur untuk memperbarui bobot model melalui gradient descent, dengan total training loss diakumulasi dan dihitung rata-ratanya di akhir setiap epoch. Progress training ditampilkan dengan mencetak nilai average training loss setiap epoch, sehingga kita dapat memantau apakah model semakin baik dalam mempelajari pola data atau tidak, yang ditandai dengan penurunan nilai loss seiring bertambahnya epoch.

1.8 Evaluasi Performa Model pada Data Testing

[11]
✓ 2 d

```
# initialize lists to monitor test loss and accuracy
test_loss = 0.0
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))

model.eval() # prep model for *evaluation*

for data, target in test_loader:
    # forward pass: compute predicted outputs by passing inputs to the model
    output = model(data)
    # calculate the loss
    loss = criterion(output, target)
    # update test loss
    test_loss += loss.item()*data.size(0)
    # convert output probabilities to predicted class
    _, pred = torch.max(output, 1)
    # compare predictions to true label
    correct = np.squeeze(pred.eq(target.data.view_as(pred)))
    # calculate test accuracy for each object class
    for i in range(batch_size):
        label = target.data[i]
        class_correct[label] += correct[i].item()
        class_total[label] += 1

# calculate and print avg test loss
test_loss = test_loss/len(test_loader.dataset)
print('Test Loss: {:.6f}\n'.format(test_loss))

for i in range(10):
    if class_total[i] > 0:
        print('Test Accuracy of %5s: %2d%% (%2d/%2d)' % (
            str(i), 100 * class_correct[i] / class_total[i],
            np.sum(class_correct[i]), np.sum(class_total[i])))
    else:
        print('Test Accuracy of %5s: N/A (no training examples)' % (classes[i]))

print('\nTest Accuracy (Overall): %2d%% (%2d/%2d)' % (
    100. * np.sum(class_correct) / np.sum(class_total),
    np.sum(class_correct), np.sum(class_total)))
```

✓

Test Loss: 0.058621

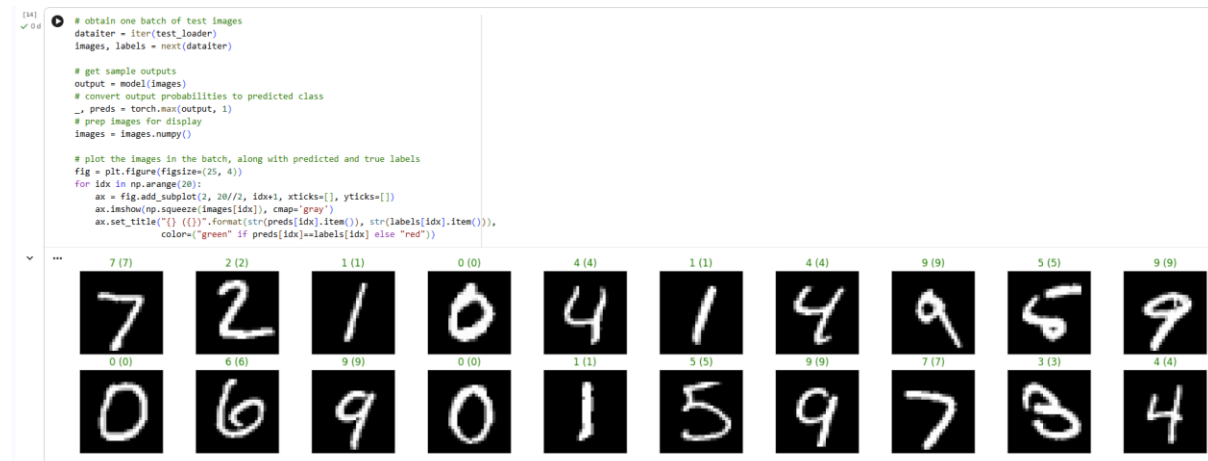
Test Accuracy of	0: 99% (972/980)
Test Accuracy of	1: 99% (1126/1135)
Test Accuracy of	2: 97% (1011/1032)
Test Accuracy of	3: 98% (991/1010)
Test Accuracy of	4: 98% (969/982)
Test Accuracy of	5: 98% (880/892)
Test Accuracy of	6: 98% (939/958)
Test Accuracy of	7: 97% (1003/1028)
Test Accuracy of	8: 96% (943/974)
Test Accuracy of	9: 97% (984/1009)

Test Accuracy (Overall): 98% (9818/10000)

Gambar 8. Evaluasi Performa Model pada Data Testing.

Kode ini merupakan tahap Evaluasi Performa Model pada Data Testing. Pada tahap ini, model yang telah dilatih diuji menggunakan test dataset untuk mengukur kemampuan generalisasinya pada data yang belum pernah dilihat sebelumnya, dengan mode evaluasi yang menonaktifkan dropout dan operasi training lainnya. Untuk setiap batch data testing, model melakukan prediksi dan hasilnya dibandingkan dengan label sebenarnya untuk menghitung akurasi per kelas (angka 0-9) dan akurasi keseluruhan, sambil juga mengakumulasi test loss. Di akhir evaluasi, statistik lengkap ditampilkan termasuk rata-rata test loss, akurasi untuk setiap digit individual, dan akurasi keseluruhan model, sehingga kita dapat menilai seberapa baik model mampu mengenali setiap angka dan performa keseluruhannya dalam mengklasifikasikan tulisan tangan pada dataset MNIST.

1.9 Visualisasi Hasil Prediksi Model



Gambar 9. Visualisasi Hasil Prediksi Model.

Kode ini merupakan tahap Visualisasi Hasil Prediksi Model. Pada tahap ini, satu batch berisi 20 gambar dari test dataset diambil dan diprediksi oleh model yang telah dilatih, kemudian hasilnya divisualisasikan dengan menampilkan gambar beserta label prediksi dan label sebenarnya untuk memverifikasi performa model secara visual. Setiap gambar ditampilkan dengan judul yang menunjukkan format "prediksi (label asli)", di mana warna hijau menandakan prediksi benar dan warna merah menandakan prediksi salah. Visualisasi ini sangat berguna untuk analisis kualitatif guna memahami jenis kesalahan yang dibuat model, seperti angka mana yang sering salah diprediksi atau pola kesalahan tertentu, sehingga dapat memberikan insight untuk perbaikan model di masa mendatang.